Network Security Groups (NSGs): Your Network Firewall

Control exactly what traffic gets in and out of every subnet and VM in Azure

🟢 Beginner | ⏱️ 15 Minutes | 📘 AZ-104 · Module 04

Why This Matters
VNets give you a private network, but how do you control what traffic gets in and out? Network Security Groups (NSGs) are like your network's firewall — they let you say "allow this traffic" or "block that traffic" at the subnet level. Without NSGs, anyone with network access could reach your databases. With them, you have complete control.

Before You Start

Prerequisites	**VNets & Subnets** (understand VNet structure first)
Time to understand	15 minutes
Difficulty	🟢 Beginner (builds on VNets, no firewall experience needed)
What you'll learn	How NSGs work, how to create rules, best practices

The Simple Idea

What Is an NSG?

An **NSG (Network Security Group)** is a **set of firewall rules that control traffic** in and out of a subnet (or a single network card on a VM).

Real-World Analogy: Bouncer at a Club

The bouncer (NSG) checks every person (traffic) trying to enter.

```
Club (Your Subnet)
│
├── Bouncer (NSG) at the door
│   ├── ✓ Allows: People with valid ID
│   ├── ✗ Blocks: People without ID
│   ├── ✗ Blocks: Troublemakers (even with ID)
│   └── Logs: Who came in and when
│
└── Inside: People dancing (your resources)
```

What NSGs Do

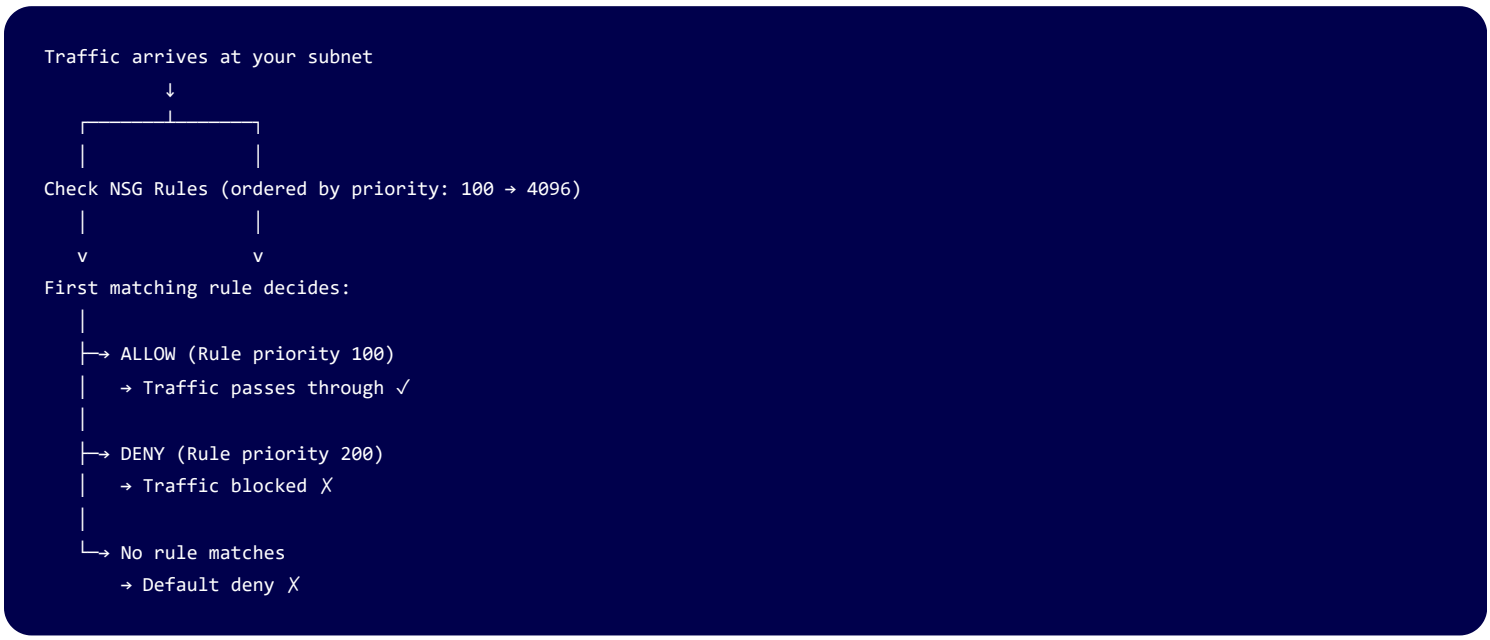
NSGs control traffic based on:

Aspect	What You Specify	Example
Source	Where traffic comes from	Internet, specific IP, another subnet
Destination	Where traffic goes to	Your VM, specific port
Port	Which port number	80 (HTTP), 443 (HTTPS), 3389 (RDP)
Protocol	TCP, UDP, or both	TCP, UDP, ICMP
Action	Allow or deny	Allow, Deny

How NSGs Work

NSG Rule Evaluation (Decision Tree)

Key insight: The **first matching rule** wins. Rules are evaluated in **priority order** (lower number = higher priority).



Default NSG Rules

Every NSG comes with **built-in default rules**:

Priority	Direction	Name	Source	Destination	Action	Purpose

65000	Inbound	AllowVNetInbound	VirtualNetwork	VirtualNetwork	Allow	Resources within VNet can talk
65001	Inbound	AllowAzureLoadBalancerInbound	AzureLoadBalancer	Any	Allow	Load balancers can send traffic
65500	Inbound	DenyAllInbound	Any	Any	Deny	Default: block everything else
65000	Outbound	AllowVNetOutbound	Any	VirtualNetwork	Allow	Resources can send to VNet
65001	Outbound	AllowInternetOutbound	Any	Internet	Allow	Resources can reach internet
65500	Outbound	DenyAllOutbound	Any	Any	Deny	Default: block everything else

Translation
Inbound: "By default, block internet traffic. Allow internal traffic."

Outbound: "By default, allow internal and internet traffic."

Mental Model: NSG as a Bouncer Checklist

```
Bouncer (NSG) has a checklist of rules:

Rule 1 (Priority 100): "Is this person on the VIP list?" → ALLOW if yes
Rule 2 (Priority 200): "Is this person on the banned list?" → DENY if yes
Rule 3 (Priority 4096): (Never reached in this example)
Default (65500): "We haven't decided yet" → DENY

Traffic arrives:
└─ Check Rule 1: Not on VIP list → Keep checking
└─ Check Rule 2: YES, banned! → DENY immediately (don't check further)
└─ Result: Bouncer blocks them at the door ✗
```

Worked Example: Real Scenario

The Scenario

TechCorp's 3-tier application:

- **Web Tier:** Should accept HTTP (port 80) and HTTPS (port 443) from internet
- **App Tier:** Should accept traffic from web tier only (port 5000)
- **DB Tier:** Should accept SQL queries from app tier only (port 1433)

Goal: Create NSG rules that enforce this security model.

Step 1: Web Tier NSG Rules

Purpose: Accept web traffic from internet, deny everything else

```
Web Subnet (10.0.1.0/24) NSG Rules:
─────────────────────────────────────

Priority 100 (HTTP from internet)
  Source: Internet (0.0.0.0/0)
  Destination: 10.0.1.0/24
  Port: 80
  Protocol: TCP
  Action: ALLOW ✓
  Purpose: Allow web traffic

Priority 110 (HTTPS from internet)
  Source: Internet (0.0.0.0/0)
  Destination: 10.0.1.0/24
  Port: 443
  Protocol: TCP
  Action: ALLOW ✓
  Purpose: Allow secure web traffic

Priority 4096 (Deny everything else)
  (This is the default deny rule)
  → Any traffic not matching rules
    100 or 110 is blocked
```

Result

```
Internet request on port 80
→ Matches Rule 100 → ALLOW ✓

Internet request on port 3389 (RDP)
→ No match → DENY ✗

Internal traffic on port 5000
→ No match → DENY ✗ (as intended)
```

Step 2: App Tier NSG Rules

Purpose: Accept traffic from web tier on port 5000, deny everything else

```
App Subnet (10.0.2.0/24) NSG Rules:
─────────────────────────────────────
```

```
Priority 100 (Traffic from web tier)
  Source: 10.0.1.0/24 (web subnet)
  Destination: 10.0.2.0/24 (app subnet)
  Port: 5000
  Protocol: TCP
  Action: ALLOW ✓
  Purpose: Allow web → app communication

Priority 110 (Default deny)
  (This is the default deny rule)
  → Any traffic not from web tier
    on port 5000 is blocked
```

Result

```
WebServer (10.0.1.5) to AppServer:5000
→ Matches Rule 100 → ALLOW ✓

Internet to AppServer:5000
→ No match → DENY ✗ (good—protected)

AppServer to Database:1433
→ Already handled by DB tier NSG
  (separate)
```

Step 3: Database Tier NSG Rules

Purpose: Accept SQL queries from app tier on port 1433, deny everything else

```
DB Subnet (10.0.3.0/24) NSG Rules:
_____________________________________

Priority 100 (SQL from app tier)
  Source: 10.0.2.0/24 (app subnet)
  Destination: 10.0.3.0/24 (db subnet)
  Port: 1433
  Protocol: TCP
  Action: ALLOW ✓
  Purpose: Allow app → database communication

Priority 110 (Default deny)
  (This is the default deny rule)
  → Any traffic not from app tier
    on port 1433 is blocked
```

Result

```
AppServer (10.0.2.10) to SQL Server:1433
→ Matches Rule 100 → ALLOW ✓

Internet to SQL Server:1433
→ No match → DENY ✗ (perfect—database
  protected)
```

```
WebServer to SQL Server:1433
→ No match → DENY ✗ (good—web can't
  bypass app)
```

Step 4: Complete Traffic Flow

```
User on internet requests web page:
└─ Traffic → WebServer (port 80)
   ├─ Web NSG rule 100 matches → ALLOW ✓
   ├─ WebServer processes request
   ├─ WebServer → AppServer (port 5000)
   │  └─ App NSG rule 100 matches → ALLOW ✓
   │  └─ AppServer processes request
   │  └─ AppServer → SQL Server (port 1433)
   │     └─ DB NSG rule 100 matches → ALLOW ✓
   │     └─ SQL Server returns data
   ├─ Data flows back through the chain
   └─ WebServer sends HTML to user ✓

SECURITY: Each tier is locked down, traffic only flows through the chain.
```

Common Mistakes (What NOT to Do)

❌ Mistake 1: Overly Permissive Rules

Wrong

```
NSG Rule:
  Source: 0.0.0.0/0 (entire internet)
  Port: 3389 (RDP - remote admin)
  Action: ALLOW

Result: Anyone on the internet can RDP
into your VM ✗
```

Fix

```
Restrict RDP to known IPs:
  Source: 203.45.67.0/24 (your office IP range)
  Port: 3389
  Action: ALLOW

Or use Azure Bastion (no public RDP needed):
  Disable port 3389 entirely on NSG
  Use Bastion to connect securely
```

Why it fails: Exposes admin interface to potential attacks.

❌ Mistake 2: Blocking Internal Communication

Wrong

```
NSG on Database Subnet:
  Source: 0.0.0.0/0 (includes internet, blocks everything)
  Port: 1433
  Action: DENY

Then try to connect from AppServer (10.0.2.10) to DB:
  ✗ DENIED (AppServer is "from somewhere"
  not explicitly allowed)
```

Fix

```
NSG on Database Subnet:
  Source: 10.0.2.0/24 (app tier specifically)
  Port: 1433
  Action: ALLOW

Now AppServer can reach database ✓
```

Why it fails: App tier can't reach database if rule is too strict.

❌ Mistake 3: Not Understanding Priority Order

Wrong

```
Priority 100: DENY port 80 from internet
Priority 200: ALLOW port 80 from internet

Traffic arrives on port 80:
  → Check priority 100: DENY immediately
  → (Priority 200 never checked because
     100 won)
  → Result: Traffic blocked ✗
```

Fix

```
Priority 100: ALLOW port 80 from internet
Priority 200: DENY port 80 from specific bad IP

Traffic from most users on port 80:
  → Check priority 100: ALLOW ✓
Traffic from malicious IP on port 80:
  → Check priority 100: Doesn't match
    (rule says 0.0.0.0/0)
  → Check priority 200: DENY ✓
```

Why it fails: First matching rule wins. The DENY came first.

❌ Mistake 4: Forgetting Default Deny

Wrong

```
NSG has no explicit rules for port 5432 (PostgreSQL)
Admin tries to access database on port 5432

"Why doesn't it work?" → Because NSG
defaults to DENY
```

Fix

```
Create explicit ALLOW rule:
  Source: Your IP or trusted network
  Port: 5432
  Action: ALLOW

Now it works ✓
```

Why it fails: No rule = blocked (by default).

NSG Rule Examples (Quick Reference)

Allow HTTP from Internet

```
Name: AllowHTTPInternet
Priority: 100
Source: 0.0.0.0/0
Destination: Any
Port: 80
Protocol: TCP
Action: ALLOW
```

Allow RDP from Your Office

```
Name: AllowRDPFromOffice
Priority: 110
Source: 203.45.67.0/24
(your office public IP)
Destination: Any
Port: 3389
Protocol: TCP
Action: ALLOW
```

Allow Traffic Between Subnets

```
Name: AllowSubnetCommunication
Priority: 120
Source: 10.0.1.0/24 (web subnet)
Destination: 10.0.2.0/24 (app subnet)
Port: 5000
```

```
Protocol: TCP
Action: ALLOW
```

Deny Telnet (Insecure Protocol)

```
Name: DenyTelnet
Priority: 200
Source: 0.0.0.0/0
Destination: Any
Port: 23
Protocol: TCP
Action: DENY
```

How This Connects to Other Topics

Related to Module 01 (Identity & Governance)

- **RBAC + NSGs:** Use RBAC to control who can modify NSGs
- **Policy + NSGs:** Enforce policies like "all subnets must have NSGs"

Related to Module 02 (Storage)

- **Storage Firewall:** Combine NSGs with storage IP rules
- **Service Endpoints:** Access storage securely without public IP

Related to Module 03 (Compute)

- **VM Security:** NSGs protect VMs from unauthorized network access
- **Web Application Firewall:** Different from NSG, works at application layer

Related to Module 05 (Monitor)

- **NSG Flow Logs:** See which traffic rules are allowing/denying
- **Network Watcher:** Diagnose connectivity issues related to NSGs

---

See It In Action

Associated lab: Lab 13: Create NSG Rules

Suggested learning sequence:

1. ✅ Read **VNets & Subnets** first
2. ✅ Read this doc (NSGs - firewall rules)
3. ✅ Read Routing Fundamentals (how traffic flows)
4. ✅ Work through Lab 13 (hands-on NSG rules)
5. ✅ Read VNet Peering (NSGs between peered VNets)

Key Takeaways

💡 **Summary**

- **NSG = Network firewall** (control inbound/outbound traffic)
- **Applied to subnets or NICs** (can protect at either level)
- **Rules evaluated by priority** (lower number = higher priority, checked first)
- **First matching rule wins** (once matched, other rules not checked)
- **Default = deny** (block everything unless explicitly allowed)
- **Think security layers:** VNet isolation + NSGs + RBAC = defense in depth
- **Always require explicit ALLOW** (deny by default is most secure)

NSG vs. Azure Firewall vs. RBAC

Security Layer	What It Controls	Best For
NSG	Network traffic (which ports/IPs)	VM-level and subnet-level firewall
Azure Firewall	Centralized traffic inspection	Enforce policies across many VNets
RBAC	Who can use Azure services	Access control at the service level

Defense in depth: All three together provide complete security.

Next Steps

1. **Learn:** Read this doc (you're here)
2. **Understand:** Read Routing Fundamentals (how traffic flows through NSGs)
3. **Practice:** Lab 13: Create NSG Rules (hands-on rule creation)
4. **Connect:** Read VNet Peering (NSGs on peered networks)
5. **Secure:** Advanced — Azure Firewall (centralized security)

Azure AZ-104
Professional Concept Documentation

Module
04 — Networking

Generated
June 15, 2026